

Notes on Bringing up the H8 Mini

*Glenn Roberts
September 2026*

In the summer of 2026 Lee Hart began developing a miniaturized version of the venerable Heathkit H8 computer, in honor of the upcoming 50th anniversary of its unveiling. To fit the H8 functionality into a small (roughly 3" x 6") form factor Lee had to vary the design from the original, most significantly by using an 8255 Programmable Peripheral Interface (PPI) chip for all input/output functions. This necessitated the development of new firmware.

I have been part of a team that did the initial work to bring this device to life, contributing primarily to development and testing of the firmware. These notes document my thoughts, techniques and lessons learned as I have gone through this process. My work has been on Revision A of the board, though Lee has already been working on Revision B.

Configuring the H8 Mini testbed

I assembled the H8 Mini (Rev. A) using Lee's instructions. The kit was supplied with a 27C256 32K EPROM, however it can be configured to use a 39SF040 (512K byte) EEPROM. Since I knew I would be doing many iterations of ROM programming I chose to use the 39SF040. As of this writing these are available from DigiKey (part #SST39SF040-70-4C-PHE-ND; \$3.87, current quantity on hand: 1,452.) To accommodate the EEPROM I used a zero-insertion force (ZIF) socket (DigiKey #A305-ND). Due to the tight spacing of chips on the board, I had to use a spare 32-lead IC socket as a spacer to provide the elevation needed to allow the ZIF socket to clear the 8255.

I set jumper P3 to "ROM" (standard setting), and P5 to "512K". Counterintuitively I had to set P4 to "32K" (not 512k) to get the EEPROM to be recognized. One other lesson learned from board construction was to face the larger opening of the piezoelectric speaker outward (the solder lug orientation suggested the opposite arrangement; however this blocks most of the sound.)

I used the SparkFun Serial Basic breakout board for the console serial line. The board is automatically powered from the SparkFun adapter so for most of my testing I did not use battery power. I used the larger raised tactile switches (DigiKey #450-1940-ND) along with 3D printed keycaps from Paul Schmidt.

Assembling the Code and Programming the EEPROM

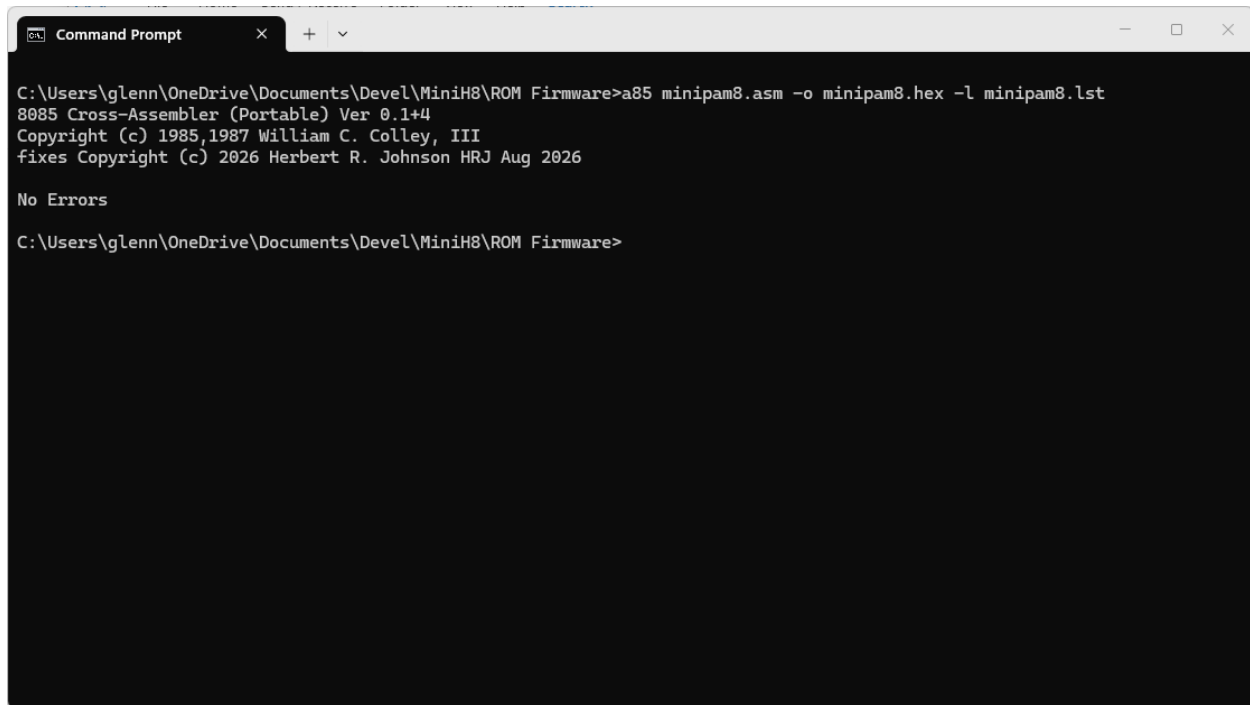
Software development was done using the A85 assembler from Herb Johnson. I ran this on my Windows 11 laptop. One useful feature that A85 seems to lack is the ability to check at time of assembly for errors in addressing. The Heath source code for PAM/8 uses the `ERRNZ` pseudo-op for this. For example, the line

```
ERRNZ    CTLFLG-MFLAG-1
```

is a way to ensure that the `CTLFLG` location is one more than the `MFLAG` location. This is a very useful feature for documenting code addressing dependencies. In this example the code uses the `HL` pair to point to `MFLAG` and then does an `INX H` to access `CTLFLG`. This can lead to subtle errors if, for example, the location of one or both of these were changed. In such a case the `ERRNZ` pseudo-op would catch that and report it.

I used the Notepad++ text editor to edit the code.

I ran the A85 assembler from a Command Prompt window:



```
Command Prompt
C:\Users\glenn\OneDrive\Documents\Devel\MiniH8\ROM Firmware>a85 minipam8.asm -o minipam8.hex -l minipam8.lst
8085 Cross-Assembler (Portable) Ver 0.1+4
Copyright (c) 1985,1987 William C. Colley, III
fixes Copyright (c) 2026 Herbert R. Johnson HRJ Aug 2026

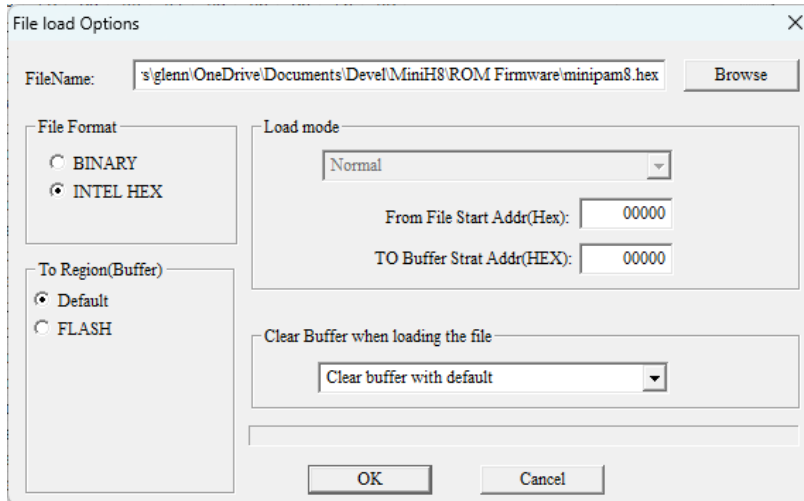
No Errors

C:\Users\glenn\OneDrive\Documents\Devel\MiniH8\ROM Firmware>
```

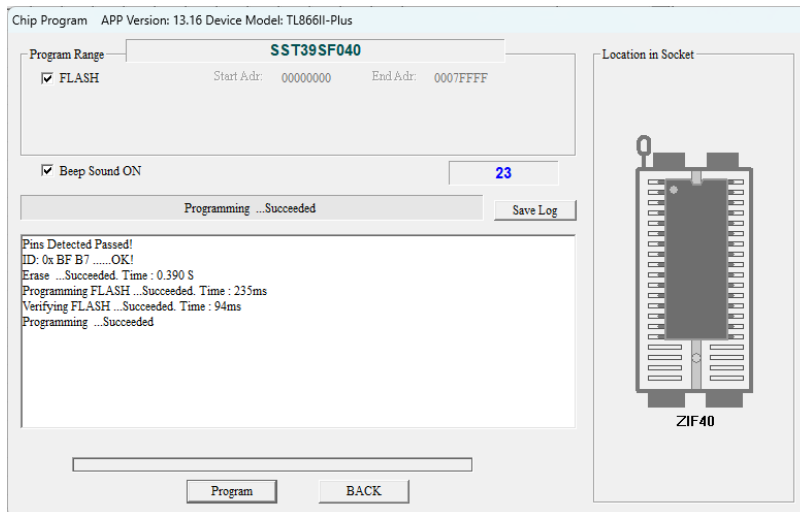
I used my TL866II+ programmer to program the EEPROM. Rebuilding the firmware and reprogramming the EEPROM took only seconds. The process was as follows:

1. Edit the code in Notepad++ and save it (Ctrl-S).
2. Use the up arrow in the Command Prompt window to recall the A85 command and reissue it to assemble the code (producing a `.LST` and `.HEX` file).

3. Insert the EEPROM in the TL866II+ programmer's ZIF socket.
4. Load the code from the Intel HEX file generated by the assembler:



5. Select "PROG" to program the EEPROM:



6. Remove the EEPROM and reinsert it in the ZIF socket on the H8 Mini, then power up the H8 Mini for testing.

As the Command Prompt tool keeps track of command history, all I had to do on subsequent assemblies was use the up-arrow key to reissue the A85 command.

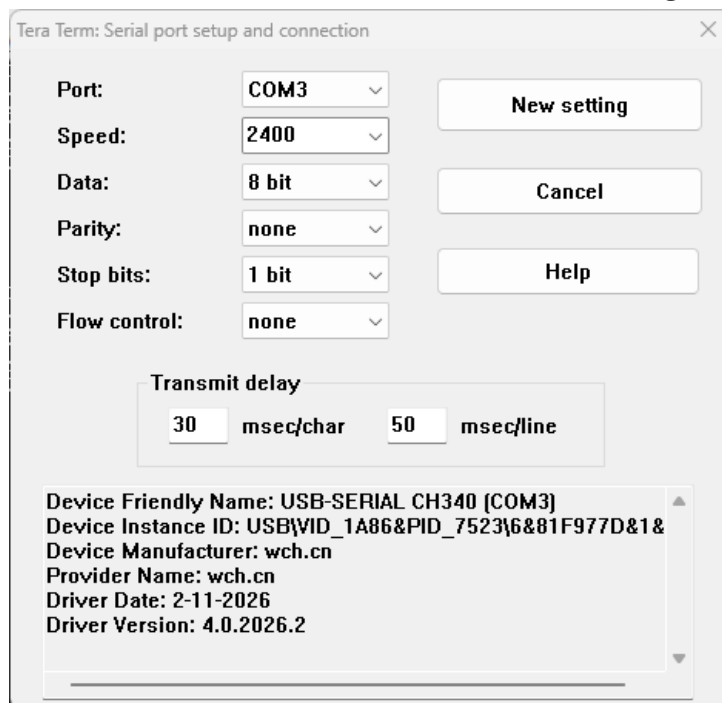
I would keep both the MINIPAM8.ASM and MINIPAM8.LST files open in Notepad++. If the assembly reported errors, I would select the MINIPAM8.LST tab (Notepad++ automatically notices that the window needs to be refreshed with the new version and prompts to do that), then scan for errors. Since error messages all begin with an uppercase letter in column 1 it is easy to search for them via the Regular Expression search feature in Notepad++. Searching for "`^[A-Z]`" finds all occurrences of an uppercase letter in column 1.

Creating MINIPAM8

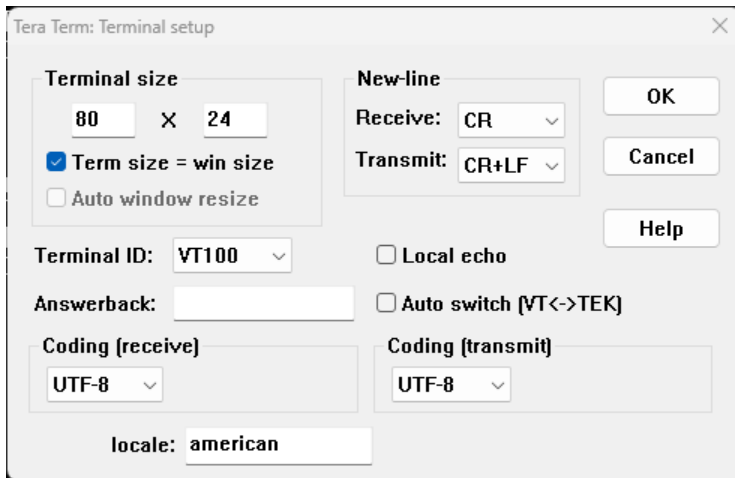
As of this writing, the creation of the new MINIPAM8 firmware is still a work in progress. I began with a version of the original Heath PAM/8 source code, which has been converted to A85 format by Herb Johnson. I verified that this code does indeed produce the exact image in the PAM/8 ROM.

The second important component was Jon Chapmans Glitch Works Monitor GWMON-80. I began with just GWMON so that I had a working console monitor. Since this monitor used bit-banged serial I found I had to tweak some of the serial timing parameters and slow things down to 2400 BAUD. These changes are documented in the code comments. With GWMON running I was able to download and run simple programs on the H8 Mini. For my console terminal I used TeraTerm version 4.105.

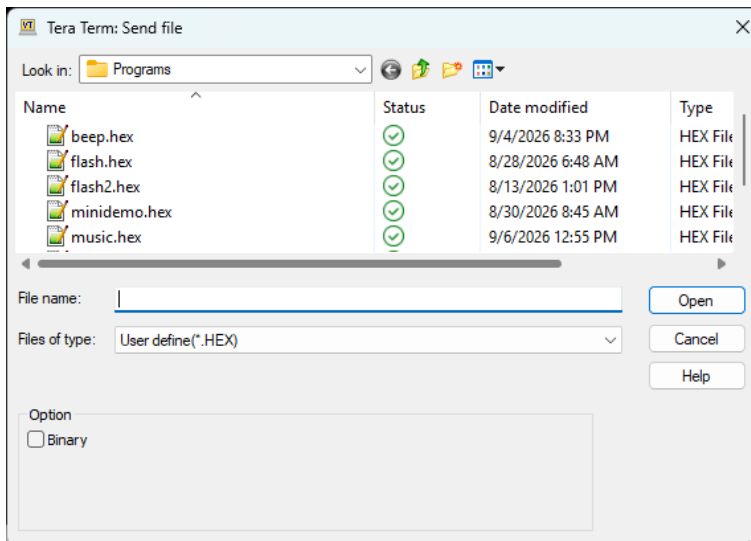
The GWMON monitor has a very useful “L” (LOAD) command, which allows you to send Intel HEX files to the H8 Mini. Here is how I configured the Serial port setup window:



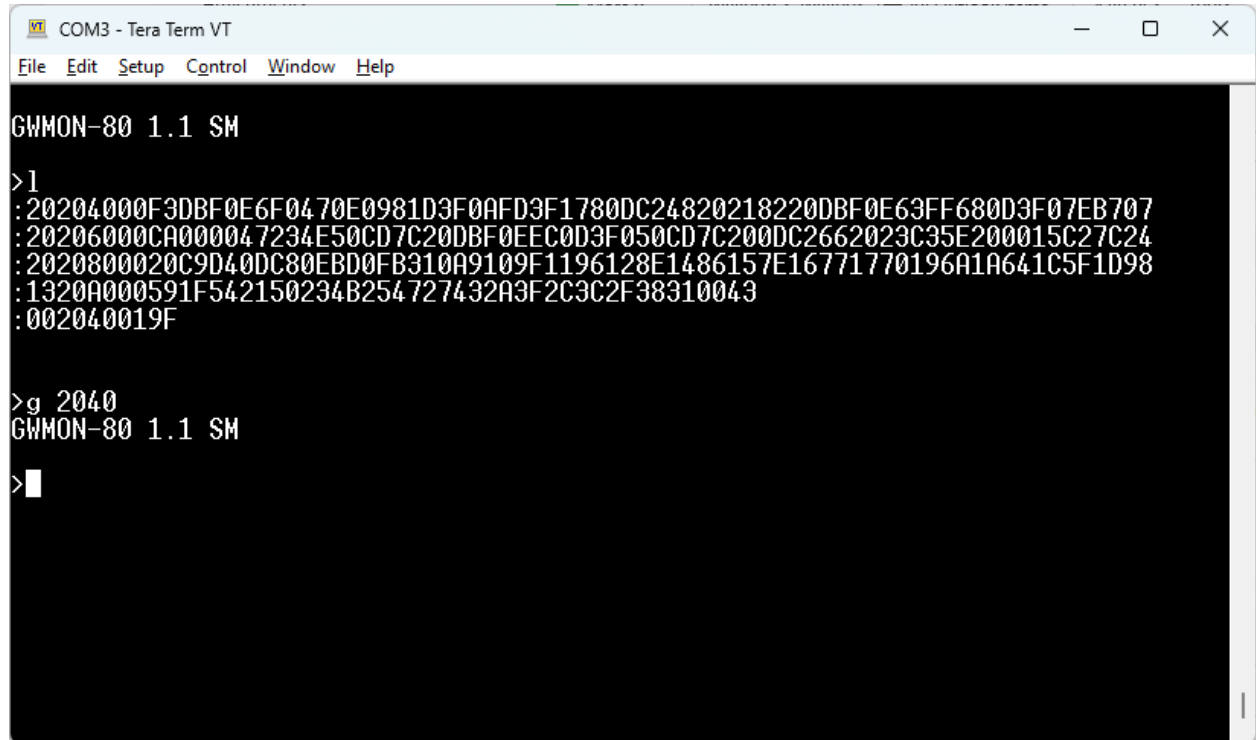
I also found that the LOAD command expects line feeds at the end of each line so I had to specify that in the terminal setup window:



I used the LOAD command to download and test software modules. This way I was able to incrementally test capabilities before folding them into the monitor. After issuing a "L" command I would invoke "File..." and "Send File..." from the TeraTerm window:



I then selected the file, hit Open and the download would proceed. The program could then be run via the monitor's "G" command:



```
COM3 - Tera Term VT
File Edit Setup Control Window Help

GWMON-80 1.1 SM

>1
:20204000F3DBF0E6F0470E0981D3F0AFD3F1780DC24820218220DBF0E63FF680D3F07EB707
:20206000CA000047234E50CD7C20DBF0EEC0D3F050CD7C200DC2662023C35E200015C27C24
:2020800020C9D40DC80EBD0FB310A9109F1196128E1486157E16771770196A1A641C5F1D98
:1320A000591F542150234B254727432A3F2C3C2F38310043
:002040019F

>g 2040
GWMON-80 1.1 SM

>|
```

Merging PAM/8 with GWMON

I have chosen to ORG PAM/8 at 0000H and GWMON at 0400H (1K). Currently the PAM/8 code provides interrupt-time processing for the front panel (e.g. LED refresh and keypad reading) and the GWMON provides the task time console functionality. Eventually PAM/8 will provide both, with an option to jump to the console via entry at 0400H.

One quirk of using the GWMON monitor while the PAM/8 interrupt processing is under way is that occasionally characters get garbled when talking to the command processor. Moving to interrupt-based serial presumably would help with this problem, however PAM/8 was designed to have the main task time executive be the front panel keypad, not a console device. My understanding of the 8085 is that the clock IRQ (RST 7.5) is a higher priority than the serial one (RST 6.5) so I'm not sure what that means for preventing clock interrupts from interfering with serial transmission? With the additions to the 2ms clock ISR that were needed to decode the keypad those interrupts now take more time to handle than on the original H8.

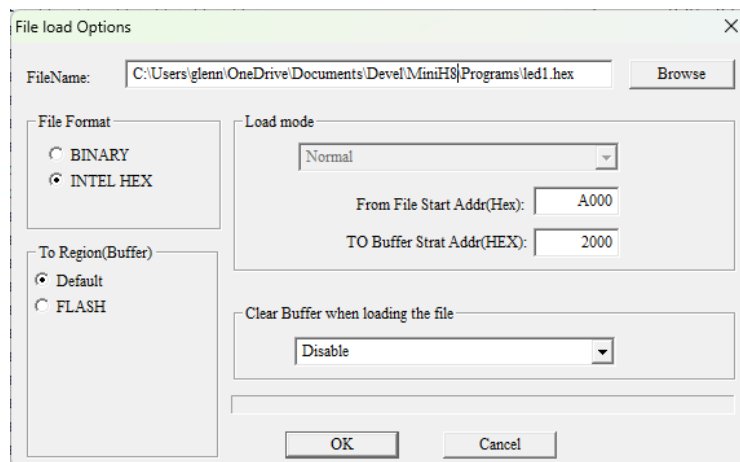
More to come here

Executing Programs from ROM

The H8 Mini memory map mirrors the entire lower 32K of ROM into the upper 32K. This means, for example, that the memory from 8000H to 9FFFH is a duplicate of the memory from 0000H to 1FFFH.

This architecture allows for up to 24K of ROM-based software to reside in memory beginning at A000H. Placing a program in this space requires a few steps:

1. ORG the program at the address from which it will be executed (e.g. A000H).
2. When programming the EEPROM first load the monitor code as normal, then layer the ROM-based code on top as follows (this example is for use with the Xgpro software):
 - a. Set the “From File Start Address” to A000H
 - b. Set the “To Buffer Start Address” to 2000H
 - c. Be sure to select “Disable” under the “Clear buffer when loading” option



3. Though your code will appear at an offset of 2000H in the EEPROM, this will show up at A000H when run from the H8 Mini.
4. You can then execute your code via G A000 in the GWMON monitor.